Multi-Model ML Benchmark

& Champion Selection Report

Automatic Block Planning System for Indian Railways

Smart India Hackathon 2026

Dataset: historical_ml_dataset.csv (1500 observations, 16 features)

Champion: XGBoost (TUNED)

Accuracy: 0.9400 | ROC-AUC: 0.9812

Generated: 2026-08-24

1. Dataset Overview

The historical ML dataset contains 1,500 asset-maintenance observations with 16 columns. The target variable (future_failure_or_critical_event_30d) is a binary indicator of whether an asset will fail or require emergency intervention within 30 days. The dataset is generated with realistic nonlinear failure physics: accelerated aging (quadratic wear), condition cliffs, hidden-defect divergence, and threshold-gated regime effects (high-traffic x poor-condition).

Feature Schema

Feature	Type	Description
asset_id	**Categorical**	**Unique asset identifier**
section_id	Categorical	Operational corridor section
department	Categorical	Engineering / Traction / S&T
task_type	Categorical	Specific maintenance task type
asset_importance	Numerical [0.50, 0.99]	Criticality weight of asset in service
asset_age	Integer [1, 30]	Years since installation
condition_score	Integer [10, 100]	Current physical condition (100=perfect)
inspection_score	Integer [10, 100]	Most recent inspection score
days_overdue	Integer [0, 40]	Days past scheduled maintenance
days_since_last_maintenance	Integer [10, 200]	Days since last service
defects_30d	Count	Defects reported in last 30 days
failures_90d	Count	Failures in last 90 days
traffic_density	Categorical	Low / Medium / High
passenger_train_count	Integer	Daily passenger trains on section
goods_train_count	Integer	Daily goods trains on section
future_failure_or_critical_event_30d	Binary {0,1}	TARGET: failure within 30 days

Target Distribution

Normal (0): 1047 (69.8%)

Failure (1): 453 (30.2%)

2. Multi-Model Accuracy Leaderboard

Six classifiers spanning linear, tree, bagging, and boosting families were trained on an 80/20 stratified split (random_state=42). Metrics are computed on the held-out test set (300 obs). Selection criterion: highest Accuracy; ROC-AUC breaks ties.

Base Models (no tuning)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Fit(s)
Gradient Boosting	**0.9300**	**0.8646**	**0.9121**	**0.8877**	**0.9823**	**0.9601**	**0.24**
XGBoost (default)	0.9400	0.8842	0.9231	0.9032	0.9809	0.9566	0.12
Random Forest	0.9333	0.8817	0.9011	0.8913	0.9776	0.9482	0.24
Logistic Regression	0.8967	0.8488	0.8022	0.8249	0.9629	0.9224	0.01
Decision Tree	0.9200	0.8317	0.9231	0.8750	0.9278	0.8364	0.00

Final Leaderboard (including XGBoost Tuned)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Fit(s)
XGBoost (TUNED)	0.9400	0.8842	0.9231	0.9032	0.9812	0.9583	0.11
XGBoost (default)	**0.9400**	**0.8842**	**0.9231**	**0.9032**	**0.9809**	**0.9566**	**0.12**
Random Forest	0.9333	0.8817	0.9011	0.8913	0.9776	0.9482	0.24
Gradient Boosting	0.9300	0.8646	0.9121	0.8877	0.9823	0.9601	0.24
Decision Tree	0.9200	0.8317	0.9231	0.8750	0.9278	0.8364	0.00
Logistic Regression	0.8967	0.8488	0.8022	0.8249	0.9629	0.9224	0.01

3. XGBoost Hyperparameter Tuning

RandomizedSearchCV (30 configurations, 3-fold stratified CV, scoring=ROC-AUC) was used to tune XGBoost. The search space covered 7 hyperparameters with 729 possible combinations.

Search Space

Parameter	Values	Description
n_estimators	**[100, 150, 250, 350]**	**Number of boosting rounds**
max_depth	[3, 4, 5, 6]	Maximum tree depth
learning_rate	[0.02, 0.05, 0.1]	Step size shrinkage
subsample	[0.7, 0.8, 1.0]	Row sampling ratio
colsample_bytree	[0.7, 0.8, 1.0]	Feature sampling ratio
min_child_weight	[1, 3, 5]	Minimum leaf weight
gamma	[0, 0.1, 0.3]	Minimum split loss

Best Hyperparameters

subsample 0.7
n_estimators 150
min_child_weight 3
max_depth 4
learning_rate 0.02
gamma 0.3
colsample_bytree 1.0

Tuning Results

Best CV ROC-AUC: 0.9654

Search time: 7.1 seconds (30 configs x 3 folds = 90 fits)

4. Champion Model: XGBoost (TUNED)

Model Name	**XGBoost (TUNED)**
Accuracy	**0.9400 (94.00%)**
Precision	0.8842
Recall (Sensitivity)	0.9231
F1 Score	0.9032
ROC-AUC	0.9812
PR-AUC	0.9583

The tuned XGBoost classifier achieves 94.00% accuracy and 0.9812 ROC-AUC on the held-out test set (300 observations). With an F1 of 0.9032 and PR-AUC of 0.9583, the model strongly discriminates high-risk assets while maintaining low false-positive rates. The tuned configuration (depth=4, lr=0.02, n_est=150) provides a balanced bias-variance tradeoff.

Confusion Matrix

Classification Report

```
precision    recall  f1-score   support

Normal (0)       0.97      0.95      0.96       209
Critical Failure (1)       0.88      0.92      0.90        91

accuracy                           0.94       300
macro avg       0.93      0.94      0.93       300
weighted avg       0.94      0.94      0.94       300
```

5. Model Comparison Summary

All six models ranked by Accuracy (selection criterion). XGBoost (TUNED) is the champion. Gradient Boosting achieved slightly higher ROC-AUC (0.9823) but lower accuracy (0.9300), so it is ranked second.

Model	Accuracy	F1	ROC-AUC	PR-AUC	Status
XGBoost (TUNED)	**0.9400**	**0.9032**	**0.9812**	**0.9583**	**CHAMPION**
XGBoost (default)	0.9400	0.9032	0.9809	0.9566	
Random Forest	0.9333	0.8913	0.9776	0.9482	
Gradient Boosting	0.9300	0.8877	0.9823	0.9601	
Decision Tree	0.9200	0.8750	0.9278	0.8364	
Logistic Regression	0.8967	0.8249	0.9629	0.9224	

Key observations:

- XGBoost benefits most from hyperparameter tuning (ROC-AUC 0.9809 -> 0.9812)
- Tree-based models dominate the leaderboard (gate-sensing advantage)
- Logistic Regression is competitive on ROC-AUC but lags on accuracy by 4.3pp
- Decision Tree underperforms due to high variance (single tree)
- Random Forest provides a strong, stable baseline (0.9776 ROC-AUC)

6. ML-Driven Criticality & Priority Assignment

The champion model predicts 30-day failure probability for each of the 1,500 historical assets. Predictions are mapped to criticality classes using calibrated probability bands, then combined with urgency (staleness since last maintenance) via the explainable 60/40 formula:

```
priority = 0.6 x criticality_weight + 0.4 x urgency
```

Criticality Probability Bands

P(failure)	Class	Weight	Interpretation
>= 0.80	**Critical**	**100**	**Imminent failure risk**
0.60 - 0.79	High	75	Elevated risk, schedule soon
0.30 - 0.59	Medium	50	Moderate risk, plan ahead
< 0.30	Low	25	Routine, safe to defer

Criticality Distribution (all 1,500 assets)

```
Critical  ####################  299 (19.9%)
    High  ########              128 (8.5%)
  Medium  #####                 87 (5.8%)
     Low  ##########################################  986 (65.7%)
```

7. Top-15 Highest-Priority Tasks

The 15 assets with the highest ML-driven priority scores. These represent the most urgent maintenance needs according to the champion model.

Asset ID	Dept	Task Type	P(fail)	Class	Urg	Priority
AST_ENG_3683	**Engineerin**	**Ballast Deep Screeni**	**0.941**	**Critical**	**100**	**100.0**
AST_TRA_4018	Traction	Neutral Section Insp	0.866	Critical	100	100.0
AST_TRA_8746	Traction	Contact Wire Sag	0.853	Critical	100	100.0
AST_TRA_7583	Traction	Contact Wire Sag	0.933	Critical	100	100.0
AST_TRA_1154	Traction	Neutral Section Insp	0.879	Critical	100	100.0
AST_ENG_3898	Engineerin	Ballast Deep Screeni	0.947	Critical	100	100.0
AST_TRA_4326	Traction	Cantilever Insulator	0.884	Critical	100	100.0
AST_ENG_2772	Engineerin	Point Machine Mechan	0.946	Critical	100	100.0
AST_TRA_7873	Traction	Cantilever Insulator	0.926	Critical	100	100.0
AST_S&T_2724	S&T	Axle Counter Error	0.918	Critical	100	100.0
AST_ENG_1940	Engineerin	Ballast Deep Screeni	0.829	Critical	100	100.0
AST_ENG_6175	Engineerin	Ballast Deep Screeni	0.877	Critical	100	100.0
AST_ENG_9694	Engineerin	Broken Rail Weld	0.931	Critical	100	100.0
AST_ENG_1046	Engineerin	Point Machine Mechan	0.946	Critical	100	100.0
AST_TRA_6366	Traction	Contact Wire Sag	0.901	Critical	100	100.0

Risk Distribution Overview

Mean P(failure): 0.303

Std P(failure): 0.362

Min P(failure): 0.019

Max P(failure): 0.949

Median P(failure): 0.061

8. Runtime Integration & Artifacts

The champion model is exported as three artifacts consumed by the production priority engine:

File	Description	Size
ml_risk_model.pkl	**Joblib serialized XGBoost classifier (tuned)**	**194 KB**
ml_model_meta.json	Model name, hyperparams, metrics, feature columns, criticality bands	1 KB
ml_asset_scores.csv	Per-asset failure probabilities, classes, and priority scores	133 KB
historical_ml_dataset.csv	Training dataset (1,500 obs, nonlinear DGP)	145 KB
generate_ml_dataset.py	Single source of truth for dataset generation	5 KB

Pipeline Flow

1. generate_ml_dataset.py -> historical_ml_dataset.csv (nonlinear DGP)
2. ml_model_comparison.ipynb -> ml_risk_model.pkl + ml_model_meta.json + ml_asset_scores.csv
3. priority_engine.py loads champion -> predicts P(failure) per DB task -> writes criticality + priority
4. optimizer_core.py consumes scored tasks -> CP-SAT solver -> optimized_schedule.csv
5. test_suite.py validates train separation, resource capacity, combined blocks, peak ban

How the Champion Drives Runtime Decisions

For each pending maintenance task in the database:

a) Simulated telemetry features are generated deterministically per asset (hash-seeded)
b) Champion XGBoost predicts P(failure in 30d)
c) Probability mapped to Criticality class via calibrated bands
d) Criticality weight combined with due-date urgency via 60/40 formula
e) Both priority_score AND criticality column updated in maintenance_tasks table
f) CP-SAT optimizer uses updated priorities + criticality for scheduling
(Critical tasks exempt from IR 12:00-18:00 peak traffic ban)
g) SHAP TreeExplainer provides per-feature attribution for any prediction

9. Conclusions

This benchmark demonstrates that a tuned XGBoost classifier is the optimal model for predicting 30-day asset failure risk in the Indian Railways maintenance context:

- Accuracy: 94.00% (highest among all 6 candidates)
- ROC-AUC: 0.9812 (near-perfect discrimination)
- F1: 0.9032 (strong precision-recall balance)
- PR-AUC: 0.9583 (robust on the imbalanced failure class)

The model exploits nonlinear failure dynamics (accelerated aging, condition cliffs, threshold-gated regime effects) that linear models cannot capture, giving it a +4.3pp accuracy advantage over Logistic Regression (0.9400 vs 0.8967).

Integration into the production pipeline ensures that maintenance blocks are prioritized by learned risk rather than static rules, maximizing asset availability while maintaining explainability through SHAP feature attributions and the transparent 60/40 scoring formula.

Artifacts exported to engine/:

ml_risk_model.pkl | ml_model_meta.json | ml_asset_scores.csv
historical_ml_dataset.csv | generate_ml_dataset.py
ml_model_comparison.ipynb (executed with embedded outputs)